

Lab 7: Linear Equations and Computational Complexity

Vladyslav Humeniuk

1 Introduction

The objective of this laboratory was to analyze the computational cost of solving systems of linear equations of the form $Ax = b$ using LU decomposition. Specifically, we aimed to empirically verify the theoretical time complexity of two distinct phases: 1. **Decomposition:** Factoring matrix A into lower (L) and upper (U) triangular matrices. Theoretical complexity: $O(N^3)$. 2. **Solving:** Using forward and backward substitution to find x . Theoretical complexity: $O(N^2)$.

2 Methodology

A C program was implemented using the GSL library to solve systems where A is a dense matrix of random values. The execution time was measured using `getrusage` (calculating user + system time) for matrix sizes N ranging from 10 to 1000.

The correctness of the solution was verified by computing the Euclidean norm of the residual vector $r = Ax - b$.

3 Results

Table 1 presents the measured execution times and the residual error for selected matrix sizes.

4 Analysis

4.1 Numerical Stability

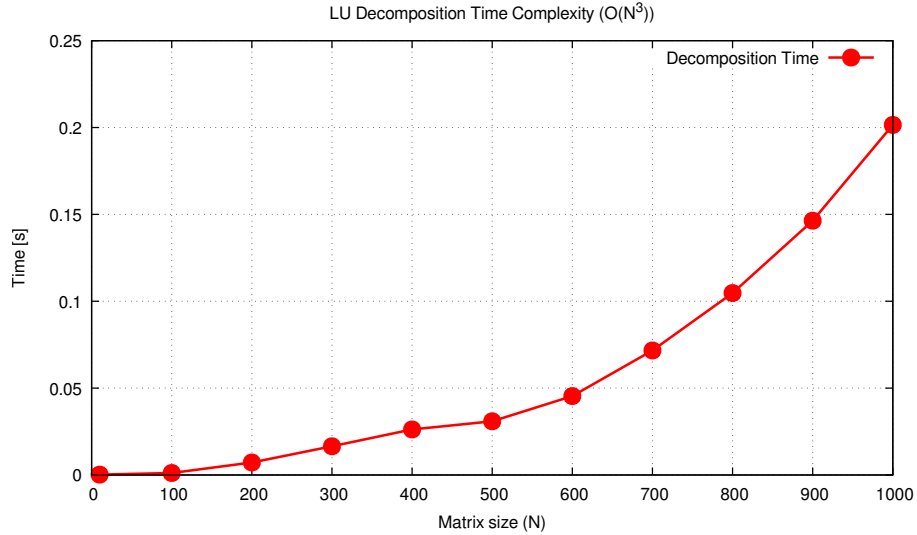
The residual error remains close to machine epsilon throughout the test range (approx. 10^{-10} for $N = 1000$), confirming that the GSL implementation of LU decomposition handles these random dense matrices correctly without significant loss of precision.

Table 1: Execution time and accuracy for varying system sizes.

N	Decomposition Time [s]	Solving Time [s]	Residual Error $\ Ax - b\ $
10	0.000206	0.000026	5.68×10^{-13}
100	0.001146	0.000061	3.93×10^{-12}
200	0.007188	0.000191	1.34×10^{-10}
300	0.016513	0.000289	5.55×10^{-11}
400	0.026210	0.000263	1.42×10^{-10}
500	0.030898	0.000285	1.83×10^{-10}
600	0.045422	0.000410	2.04×10^{-09}
700	0.071670	0.000560	3.23×10^{-10}
800	0.104717	0.000754	2.00×10^{-10}
900	0.146367	0.000982	5.49×10^{-10}
1000	0.201480	0.001222	1.20×10^{-09}

4.2 Time Complexity

Decomposition ($O(N^3)$): Figure 1 illustrates the time required to factorize the matrix. The growth is clearly non-linear. Comparing $N = 500$ (0.0309 s) to $N = 1000$ (0.2015 s), the time increases by a factor of approximately 6.5. While theoretical cubic growth implies an $8\times$ increase (2^3), the deviation is likely due to CPU cache efficiency being higher for smaller matrices and overhead dominance at lower N .

Figure 1: LU Decomposition time as a function of matrix size N .

Solving ($O(N^2)$): Figure 2 shows the time for the back-substitution phase. Comparing $N = 500$ (0.000285 s) to $N = 1000$ (0.001222 s), the time increases by a factor of 4.3. This aligns almost perfectly with the theoretical quadratic complexity ($2^2 = 4$).

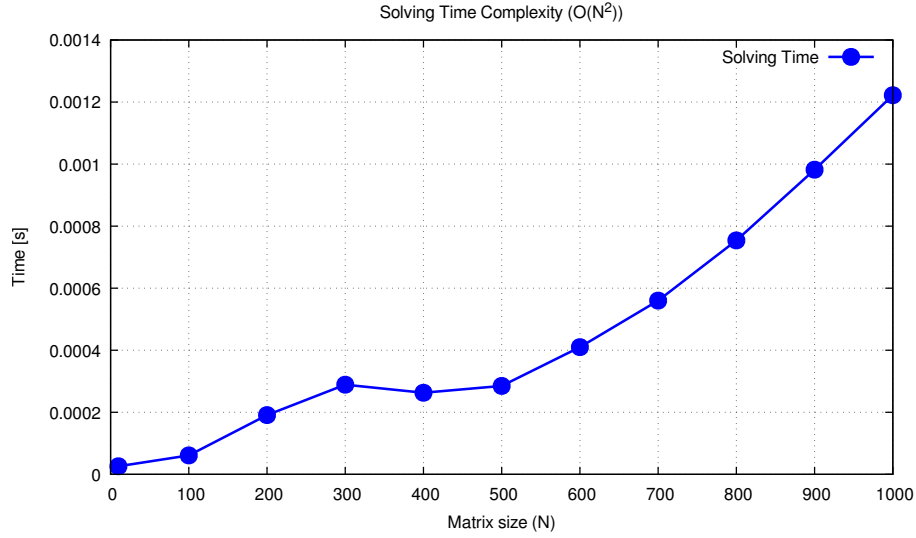


Figure 2: Solving time (forward/backward substitution) as a function of matrix size N .

4.3 Comparison

At $N = 1000$, decomposition takes 0.20s while solving takes only 0.0012s. The decomposition is roughly **165 times more expensive** than the solving step. This highlights the efficiency of LU decomposition in scenarios where the system $Ax = b$ must be solved for many different vectors b while A remains constant. The computationally expensive factorization is performed once, allowing for rapid calculation of subsequent solutions.

5 Conclusion

[cite_start]The experiment successfully verified the theoretical complexity classes of linear algebra operations[cite: 4, 5]: 1. **LU Decomposition** follows an $O(N^3)$ trend, dominating the total execution time for large systems. 2. **Back-substitution** follows an $O(N^2)$ trend. 3. The numerical error remains negligible for the tested range.

Separating the factorization from the solving phase is a critical optimization for simulations requiring repeated solutions of the same physical system under different load conditions.

Bibliography